

RCC Onboarding - Part 3

Performance, CPU and GPU resources, memory, storage, and efficient I/O

IKIM RCC documentation proposal

11 July 2026

Contents

Purpose and learning goals

Information that administrators must complete

Why performance design matters

The bottleneck model

CPU performance and parallelism

GPU performance and suitability

RAM and working sets

Storage locations and disk space

I/O fundamentals: latency, throughput, IOPS, and metadata

Streaming versus random I/O

Large files, small files, and crowded directories

Compression and biomedical file formats

Good I/O patterns with Snakemake and node-local storage

Measuring jobs and finding bottlenecks

Performance experiments and scaling

Common anti-patterns

Completion checklist

References for maintainers

Purpose and learning goals

Part 3 is for biomedical researchers who can already submit Slurm jobs and run a Snakemake workflow. It explains why the same scientific calculation can finish in hours or continue for months depending on resource requests, file layout, software settings, and data movement.

By the end of Part 3, you should be able to:

distinguish CPU, GPU, RAM, local disk, shared storage, and network I/O;
 identify the most likely bottleneck in a job;
 request resources that match the program rather than guessing;
 recognise harmful small-file and random-I/O patterns;
 keep compressed biomedical data compressed when tools support it;
 use Snakemake's temporary-directory, shadow, benchmark, and resource mechanisms;
 stage I/O-intensive work to node-local storage; and
 use Slurm and Linux measurements to improve a workflow systematically.

Central performance principle A job is limited by its slowest required resource. Adding CPU cores does not accelerate a job that is waiting for network storage. Adding RAM does not accelerate a single-threaded program. Requesting a GPU does not help software that has no GPU implementation.

Information that administrators must complete

Replace every value marked **ADMIN** before publication.

Node-local scratch path or variable: [ADMIN: document \$SLURM_TMPDIR, \$TMPDIR, or the RCC-supported equivalent]

Scratch capacity and cleanup policy: [ADMIN: state quotas, lifetime, and automatic deletion behavior]

CPU partitions and limits: [ADMIN: list names, maximum time, and core limits]

GPU partitions, GPU types, and request syntax: [ADMIN: list supported Slurm resources and constraints]

Memory limits and high-memory partitions: [ADMIN: document site policy]

Shared project-storage guidance: [ADMIN: state quotas, snapshots, backup status, and intended use]

Job-accounting fields available on RCC: [ADMIN: verify sacct, sstat, and whether seff is installed]

I/O monitoring tools available to users: [ADMIN: verify iostat, pidstat, iotop, strace, and perf]

Recommended maximum files per directory: [ADMIN: provide an RCC operational threshold]

Support contact: [ADMIN: insert RCC support address]

1. Why performance design matters

Correct science can still be computationally impractical

A workflow may be scientifically correct but operationally poor. Consider 1,000 sequencing samples. If every sample starts a program 20 times, repeatedly scans a directory containing 100,000 files, and performs millions of small random reads over shared storage, the cluster spends much of its time waiting rather than calculating.

A teaching example illustrates the scale of the problem:

an efficient rule processes one sample in 30 minutes;

an inefficient file layout and repeated decompression make the same rule take 12 hours;

1,000 samples at 30 minutes require 500 compute-hours;

1,000 samples at 12 hours require 12,000 compute-hours.

If only a small number of jobs can run concurrently, the difference can turn a multi-day project into a multi-month project. These numbers are illustrative, not RCC benchmark results. The principle is real: small inefficiencies multiply across samples, rules, and retries.

Performance is part of reproducibility

Record resource requirements and measured performance with the workflow. A result that can be reproduced only by requesting an entire large-memory node for every rule is not operationally mature. Good workflows state what each rule needs and provide evidence from benchmark data.

2. The bottleneck model

A pipeline is a chain of constrained stages

A biomedical workflow commonly includes:

compressed input -> read/parse -> compute -> temporary data -> write result

Each stage can be limited by a different resource:

Resource	What it provides	Typical bottleneck symptom
CPU	Executes instructions	One or more cores remain near full utilisation
GPU	Executes supported massively parallel kernels	GPU utilisation is high and the application is GPU-enabled
RAM	Holds active data structures and caches	Memory use approaches the request; swapping or OOM occurs
Local disk	Fast temporary workspace on one node	Job is fast locally but slow on shared storage
Shared storage	Durable project data accessible to many nodes	Many jobs wait on reads, writes, or metadata operations
Network	Moves data between storage and nodes	Throughput plateaus; additional jobs reduce per-job speed

Utilisation is not the same as efficiency

A CPU may be busy performing avoidable decompression. A GPU may show low utilisation because the CPU cannot prepare batches quickly enough. A disk may be busy serving thousands of tiny metadata requests while transferring very little useful data. Measure useful progress as well as utilisation.

Wall time, CPU time, and throughput

Wall time is the elapsed time observed by the researcher.

CPU time is the sum of time used by CPU cores. Eight fully used cores for one hour can produce approximately eight CPU-hours.

Throughput is useful work per unit time, such as samples per hour or gigabytes processed per hour.

Latency is the delay before an operation completes, which is especially important for many small operations.

Use a scientific throughput metric that matches the project. For example: samples/hour, reads/second, variants/minute, images/second, or model batches/second.

3. CPU performance and parallelism

Cores, threads, processes, and tasks

A physical CPU contains multiple cores. Software can use them through processes or threads. Slurm must reserve the cores before the program uses them.

For a single multithreaded program, the common Slurm model is:

```
#SBATCH --ntasks=1
```

```
#SBATCH --cpus-per-task=8
```

The program must also be told to use eight threads. Examples include `samtools sort -@ 8`, `fastp --thread 8`, or a Python/R library configured for eight threads. Reserving eight cores while running a one-thread command wastes seven cores. Running eight threads after requesting one core oversubscribes the node and interferes with other jobs.

More cores do not guarantee proportional speed

Parallel speedup is limited by:

- serial parts of the program;
- synchronisation between threads;
- memory bandwidth;
- input and output speed;
- algorithm design; and
- the size of the problem.

A program may improve from one to four threads and show almost no improvement from eight to sixteen. Measure representative data at 1, 2, 4, 8, and possibly 16 threads. Choose the smallest request near the useful performance plateau.

Parallelism has two levels

Across samples: Snakemake can run independent samples as separate Slurm jobs.

Within a sample: one program may use several threads.

For many biomedical workflows, moderate threading per sample plus several samples in parallel is more efficient than giving every sample the maximum number of cores.

Avoid hidden thread multiplication

Numerical libraries may create threads through OpenMP, BLAS, MKL, NumExpr, TensorFlow, PyTorch, or Java. A Python process can therefore use more CPU cores than expected.

Inside Slurm jobs, consider site-approved settings such as:

```
export OMP_NUM_THREADS="$SLURM_CPUS_PER_TASK"
export OPENBLAS_NUM_THREADS="$SLURM_CPUS_PER_TASK"
export MKL_NUM_THREADS="$SLURM_CPUS_PER_TASK"
```

Do not set these globally without understanding the application. Some programs manage threading themselves.

4. GPU performance and suitability

A GPU is a specialised accelerator

GPUs are effective when software implements large, parallel numerical operations. Typical biomedical examples include deep-learning training and inference, some image-processing methods, molecular simulation, selected base-callers, and selected sequence-analysis tools.

A GPU does not automatically accelerate:

- shell scripts;
- ordinary file copying;
- most compression utilities;
- tools compiled without GPU support;
- small tasks dominated by startup or I/O; or

algorithms with substantial serial control flow.

Request a GPU only for a GPU-enabled rule

RCC syntax is site-specific. A typical Slurm request may resemble:

```
#SBATCH --gres=gpu:1
```

or may use newer GPU options. Use only the syntax published by RCC administrators.

The job still needs CPU cores and RAM. The CPU prepares data, runs preprocessing, coordinates kernels, and performs I/O. GPU memory is separate from system RAM.

GPU bottlenecks

Observation	Likely interpretation
GPU utilisation near zero	Program is not using the GPU, is waiting for data, or is between kernels
GPU memory full but compute low	Model/batch occupies memory but input or CPU preparation is limiting
GPU compute high and stable	GPU is likely doing useful work; verify scientific throughput
CPU saturated, GPU intermittent	CPU preprocessing or decompression may be limiting
Long startup before GPU activity	Environment, container, data staging, or model loading may dominate

Useful commands, when available, include:

```
nvidia-smi
```

```
nvidia-smi dmon -s pucvmet
```

Run monitoring commands within the allocated job or through an approved interactive allocation, not on a general login host.

5. RAM and working sets

RAM is not disk space

RAM holds data that a running program needs immediately. Disk stores files persistently. A 100 GB input file does not necessarily require 100 GB RAM because many tools stream through it. Conversely, an assembler may require far more RAM than the compressed input size because it builds large in-memory graphs.

Resident memory and virtual memory

RSS or resident set size approximates memory physically held in RAM.

Virtual memory includes address space that may not be resident and is often less useful for routine sizing.

MaxRSS from Slurm accounting is commonly used to estimate peak resident memory, subject to site accounting configuration.

Too little memory

When a job exceeds its enforced memory limit, Slurm or the operating system may terminate it. Logs may mention `OUT_OF_MEMORY`, `oom-kill`, `Killed`, or exit code 137.

Do not solve every failure by requesting the largest possible memory. First check whether:

the program loaded unnecessary data;

the number of threads increased memory use;

temporary files were placed in RAM-backed storage;

an input was malformed;
the algorithm has a memory-saving mode; or
samples can be processed separately.

Too much memory also has a cost

Large memory requests may wait longer in the queue and reduce cluster utilisation. Start from measured representative jobs and include a safety margin appropriate to variability.

6. Storage locations and disk space

Different storage has different purposes

Location	Intended use	Lifetime	Typical performance
Laptop	Local editing and approved small files	User-managed	Local, not cluster-accessible
Home	Configuration, code, small documents	Persistent, usually quota-limited	Shared; not for large datasets or software trees unless approved
Project storage	Approved inputs and durable results	Persistent according to project policy	Shared across nodes
Node-local scratch	Temporary working files for one job	Deleted after job or node cleanup	Usually low-latency and high-throughput
Archive	Infrequently accessed retained data	Long-term	Optimised for capacity, not active computation

Confirm actual RCC paths and policies before publication.

Capacity planning

Estimate more than final output size. Many tools create temporary files, indexes, sorted copies, intermediate alignments, caches, and logs. A safe estimate considers:

peak disk requirement = inputs copied locally
+ temporary files
+ outputs before transfer
+ safety margin

Snakemake supports standard disk or disk_mb resources. These express a requirement to the execution backend; they do not by themselves delete files or guarantee that a site has local scratch.

Inspect space and file counts

```
df -h /path/to/filesystem
du -sh project/
du -sh project/* | sort -h
find project -type f | wc -l
```

Use du carefully on very large directory trees because walking millions of files itself creates metadata load.

7. I/O fundamentals: latency, throughput, IOPS, and metadata

I/O means moving data

Input/output includes reading and writing file contents, opening files, listing directories, checking permissions, reading metadata, renaming files, and deleting files.

Four concepts matter:

Throughput: bytes transferred per second, often MB/s or GB/s.

Latency: time required for one operation to begin or complete.

IOPS: number of input/output operations per second.

Metadata operations: operations on file names, directories, permissions, and attributes rather than file content.

Shared network storage can deliver excellent streaming throughput while still being slow for millions of small or random operations.

The storage is shared

A network filesystem is a collective resource. A workflow that creates a metadata storm can slow unrelated users. Good I/O behavior is therefore both a performance requirement and cluster etiquette.

8. Streaming versus random I/O

Streaming or sequential I/O

Streaming reads a file in order, usually in large blocks:

start -> block 1 -> block 2 -> block 3 -> end

Examples include reading a compressed FASTQ once, writing a BAM sequentially, or scanning a CSV table. Storage and operating-system read-ahead can optimise this pattern.

Random I/O

Random I/O jumps repeatedly between positions:

block 900 -> block 4 -> block 42000 -> block 7 -> ...

Examples include an algorithm performing many small indexed reads, a database with scattered pages, or thousands of processes reading unrelated small environment files.

Random I/O is especially harmful over network storage because each operation pays latency and may require network and metadata-server coordination. A local NVMe device can handle far more random operations than a shared filesystem, though it is still finite.

Transform random network I/O into local I/O

When a rule performs intensive random access:

stage required inputs to node-local storage;

run the random-access calculation locally;

validate outputs;

copy only durable results back to project storage; and

allow job-local temporary data to be deleted.

Snakemake mechanisms for this pattern are covered in Section 11.

9. Large files, small files, and crowded directories

Large streaming files are usually friendlier than many tiny files

One 10 GB file and one million 10 KB files contain similar amounts of data, but the million-file case requires vastly more opens, closes, path lookups, permission checks, directory entries, and metadata updates.

Common biomedical small-file sources include:

per-cell or per-feature intermediate files;
 one JSON file per task;
 extracted Conda environments;
 thousands of temporary chunks;
 per-read or per-region outputs; and
 workflow logs without hierarchical organisation.

Avoid hundreds of thousands of entries in one directory

Large flat directories are difficult for users and can stress directory operations. Prefer a hierarchy such as:

```
results/
  project_batch_01/
    sample_0001/
    sample_0002/
  project_batch_02/
    sample_1001/
```

Other strategies include:

group by cohort, sequencing run, assay, sample prefix, or processing stage;
 maintain a manifest table mapping identifiers to paths;
 aggregate tiny tabular outputs into Parquet, HDF5, Zarr, SQLite, or another appropriate scientific format;
 package inactive small files into a tar archive for transfer or archive; and
 avoid unpacking archives merely to inspect one file when tools can stream or select members.

Do not use one shared writable archive as an active parallel-workflow filesystem. Archives are appropriate for transfer and cold storage, not concurrent random updates.

10. Compression and biomedical file formats

Leave files compressed when the tool supports them

Compression reduces storage consumption and network traffic. Many bioinformatics tools directly read .fastq.gz, .vcf.gz, and other compressed formats. Decompressing them permanently may multiply project storage and I/O without improving the analysis.

Good practice:

preserve original compressed FASTQ files;
 use tools that read compressed input directly;
 use BGZF/block-gzip formats when indexed random access is required, for example compressed VCF with an index;
 write compressed BAM/CRAM rather than SAM for durable alignments;
 avoid repeatedly decompressing and recompressing the same data between rules; and
 use streams or pipes between compatible tools when this removes a large temporary file and error handling remains robust.

Example:

```
minimap2 -t 8 reference.fa reads_R1.fastq.gz reads_R2.fastq.gz \
| samtools sort -@ 8 -T "$TMPDIR/sample" -o sample.bam
```

This avoids writing a large uncompressed SAM file. It does not eliminate the need to check exit codes and logs. Use set -o pipefail in shell scripts; Snakemake's Bash strict mode also helps detect pipeline failures.

Compression uses CPU

Compression is a trade-off: fewer bytes moved, more CPU work. On a network-limited workflow, compression often improves overall time. On a CPU-limited workflow with very fast local storage, compression level may need tuning. Measure the complete pipeline rather than one command.

11. Good I/O patterns with Snakemake and node-local storage

Use the standard tmpdir resource

Snakemake treats tmpdir as a standard resource and sets \$TMPDIR for shell commands, scripts, wrappers, and notebooks. The RCC profile should map it to the job's node-local temporary directory.

An administrator-managed profile may contain a site-specific setting such as:

```
executor: slurm
default-resources:
  mem_mb: 4000
  runtime: 60
  disk_mb: 10000
  tmpdir: system_tmpdir
```

On RCC, system_tmpdir must resolve to the site-supported local scratch through \$TMPDIR, \$SLURM_TMPDIR, or equivalent configuration.

A rule should direct tool-specific temporary files to \$TMPDIR:

```
rule sort_bam:
  input:
    "results/{sample}.unsorted.bam"
  output:
    "results/{sample}.bam"
  threads: 8
  resources:
    mem_mb=16000,
    disk_mb=80000,
    runtime=120
  benchmark:
    "benchmarks/sort_bam/{sample}.tsv"
  shell:
    """
    samtools sort \
      -@ {threads} \
      -T "$TMPDIR/{wildcards.sample}" \
      -o {output} \
      {input}
    """
```

The durable output remains on project storage, while large temporary sort files are local.

Use shadow for isolated rule execution

A shadow rule runs in an isolated directory. With an RCC profile setting shadow-prefix to node-local scratch, temporary files created with relative paths remain local.

rule summarise_large_intermediate:

```
input:
  "data/{sample}.tsv.gz"
output:
  "results/{sample}.summary.tsv"
shadow:
```

```

"copy-minimal"
benchmark:
  "benchmarks/summarise/{sample}.tsv"
shell:
  r"""
  analysis_tool {input} --temporary huge.tmp --output {output}
  """

```

minimal links declared inputs into the shadow directory.

copy-minimal copies declared inputs, useful when reads must also be local.

shallow and full reproduce more of the work-directory structure.

Use copy-minimal selectively. Copying a multi-terabyte input for a short streaming rule can be slower than reading it once from shared storage. It is most useful when the program performs repeated or random reads and the local-copy cost is smaller than the avoided network I/O.

Use temporary outputs deliberately

Snakemake's temp() marker tells Snakemake that an intermediate can be removed after downstream consumers finish:

```

output:
  temp("intermediate/{sample}.bam")

```

Do not mark the only scientifically required copy as temporary. Keep raw inputs, final outputs, provenance, and required quality-control reports according to project policy.

Limit concurrent I/O-heavy jobs

Submitting hundreds of data-intensive jobs simultaneously may reduce total throughput. Define an artificial resource and set a workflow-wide limit:

```

rule io_heavy_step:
  resources:
    io_slots=1

```

```
snakemake --resources io_slots=8 ...
```

The correct limit must be measured and may differ by dataset and storage system.

Avoid needless existence checks and huge DAGs

A workflow that inventories millions of files at startup can spend substantial time on metadata. Use manifests, predictable paths, and sensible aggregation. Do not repeatedly call glob or find over large shared trees in every rule.

12. Measuring jobs and finding bottlenecks

Start with Slurm accounting

After a job completes, inspect fields available at RCC:

```
sacct -j JOBID \
--
```

```
format=JobID,JobName,State,Elapsed,AllocCPUS,TotalCPU,MaxRSS,AveDiskRead,AveDiskWrite,ExitCode
```

Interpret cautiously:

Elapsed is wall time.

AllocCPUS is the CPU allocation.

TotalCPU is accumulated user plus system CPU time.

MaxRSS is peak resident memory when accounting supports it.

disk fields depend on the accounting plugin and may not represent network-filesystem traffic completely.

For a running job, the site may support:

```
sstat -j JOBID.batch --format=JobID,AveCPU,MaxRSS,AveDiskRead,AveDiskWrite
```

Accounting availability depends on RCC configuration.

Estimate CPU efficiency

A simple approximation is:

CPU efficiency = TotalCPU / (Elapsed x AllocCPUS)

Near 100% suggests allocated CPU cores were busy. Low values may indicate I/O waits, serial code, idle phases, GPU waiting, or over-requested cores. It does not prove scientific efficiency.

Snakemake benchmarks

Add a benchmark file to important rules:

benchmark:

```
"benchmarks/{rule}/{sample}.tsv"
```

Snakemake records timing and resource information. Use repeated representative runs and retain benchmark summaries under version control or in a results report, not necessarily every raw benchmark file forever.

Linux tools

Availability varies by site:

Tool	Question it helps answer
/usr/bin/time -v	What wall time, CPU time, and peak memory did one command use?
top or htop	Are CPU cores and memory active now?
free -h	How much node memory is available?
vmstat 1	Is the node CPU-bound, waiting on I/O, or swapping?
iostat -xz 1	Are local block devices saturated?
pidstat -dru 1	Which process uses CPU, memory, and I/O?
nvidia-smi dmon	Is a GPU active and memory occupied?
strace -c	Is a program spending time in many file/system calls? Use only for targeted debugging.
perf stat	What CPU events occur? Access may be restricted.

Do not run high-frequency monitoring continuously across many jobs. Monitoring itself consumes resources and creates logs.

Recognise common signatures

High CPU efficiency, runtime improves with cores: CPU-bound.

Low CPU efficiency, high read/write volume, cores mostly idle: I/O-bound.

MaxRSS close to request or OOM: memory-bound or under-requested.

GPU low, CPU high: preprocessing or data-loader bottleneck.

GPU high, CPU moderate: likely GPU-bound.

Many seconds before program starts: environment activation, container startup, metadata, or data staging.

Performance collapses when many jobs run: shared I/O or service contention.

13. Performance experiments and scaling

Change one factor at a time

Use one representative sample and record:

input size and checksum;

software/container version;

parameters;

threads and memory request;

local versus shared data location;

elapsed time;

CPU time and MaxRSS;

bytes read/written when available; and

scientific throughput.

Then vary one factor: thread count, compression level, batch size, scratch use, or number of concurrent samples.

Strong and weak scaling

Strong scaling: same problem, more resources. Does it finish faster?

Weak scaling: larger problem with proportionally more resources. Does time remain stable?

For routine biomedical workflows, a simple thread-scaling table and an across-sample concurrency test are often sufficient.

Optimise the largest contributors first

Use the workflow DAG and benchmark totals. Optimising a rule that consumes 1% of total runtime cannot reduce overall runtime by more than approximately 1%. Focus on rules that dominate elapsed time, compute-hours, storage traffic, or failure frequency.

14. Common anti-patterns

Running many tiny jobs

Thousands of jobs that run for seconds create scheduler and filesystem overhead. Group tiny rules or process sensible batches where scientific independence and failure recovery permit.

Requesting every available core

A tool that scales to four threads may become slower at 64 because of memory bandwidth and I/O contention. Measure scaling.

Reading the same remote data repeatedly

Repeated reference scans, repeated decompression, and repeated random access multiply network traffic. Reuse indexes, stream once, cache appropriately, or stage to local scratch.

Creating one file per observation

One file per cell, feature, read, or genomic interval can create millions of files. Use suitable container formats and manifests.

Permanently decompressing FASTQ data

Most sequence tools read gzip-compressed FASTQ. Permanent decompression consumes space and I/O. Decompress only when a tool requires it, preferably in node-local scratch.

Putting temporary files on project storage

Sort files, decompressed intermediates, compiler caches, and application scratch should use node-local temporary storage when possible.

Ignoring partial outputs after failure

Write to temporary paths and let Snakemake move validated outputs into place. Use Snakemake's incomplete-file handling, logs, and checksums where appropriate.

Using network storage for software environments

Conda environments can contain tens of thousands of small files. Starting many jobs from those environments creates random and metadata-heavy I/O. Part 4 explains why RCC prefers Apptainer SIF images for deployed workflow software.

15. Completion checklist

You can complete Part 3 when you can:

- explain the difference between CPU, GPU, RAM, disk capacity, and I/O;
- state whether a representative rule is CPU-, GPU-, memory-, or I/O-limited;
- match threads: to the program's thread option and Slurm allocation;
- use sacct and a Snakemake benchmark file;
- explain streaming versus random I/O;
- explain why many small files are expensive on shared storage;
- keep supported sequence files compressed;
- direct temporary files to \$TMPDIR;
- describe when shadow: "copy-minimal" is useful and when it is wasteful; and
- show a measured performance comparison rather than relying on intuition.

References for maintainers

Snakemake 9.23.1, Snakefiles and Rules: standard resources, tmpdir, benchmarks, and shadow execution.

Snakemake 9.23.1, Command Line Interface: profiles, local-storage prefixes, and shared-filesystem assumptions.

Snakemake 9.23.1, FAQ: node-local storage with shadow and --shadow-prefix.

Slurm, sbatch, sacct, and sstat manuals.

NVIDIA, nvidia-smi documentation.

The RCC-specific profile, scratch paths, partitions, and accounting fields must be validated on production before publication.